



Deep One-Class Classification

*Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deecke, Shoaib
A. Siddiqui, Alexander Binder, Emmanuel Müller, Marius Kloft*

By Vedanshi Shah, Abbie Murphy, Jimmy Kong

START HERE

Authors

Deep One-Class Classification



Lukas Ruff

Aignostics

Bestätigte E-Mail-Adresse bei aignostics.com - [Startseite](#)

[Machine Learning](#) [Deep Learning](#) [Trustworthy ML](#) [Anomaly Detection](#) [Digital Pathology](#)



Shoaib Ahmed Siddiqui

[University of Cambridge](#)

Verified email at cam.ac.uk

[Artificial Intelligence](#) [Deep Learning](#)



Nico Goernitz

Freelancer | CTO | ex-postdoc@TU-Berlin

Verified email at tu-berlin.de - [Homepage](#)

[Artificial Intelligence](#) [Machine Learning](#) [Anomaly Detection](#) [One-class Learning](#)



Alexander Binder

Chair Computer Vision, [OvGU](#), Magdeburg, Germany

Bestätigte E-Mail-Adresse bei ovgu.de

[Explainable Deep Learning](#) [XAI](#) [Aspects of Machine Learning](#)



Robert A. Vandermeulen

Independent Researcher

Verified email at umich.edu - [Homepage](#)

[Machine Learning](#) [Deep Learning](#) [Nonparametric Statistics](#) [Nonparametric Tensor Meth...](#)



Emmanuel Müller

Professor of Computer Science, Technical University of Dortmund

Verified email at cs.tu-dortmund.de - [Homepage](#)

[Data Mining](#) [Machine Learning](#) [Data Exploration](#) [Databases](#)



Lucas Deecke

[University of Edinburgh](#)

Verified email at ed.ac.uk

[Machine Learning](#) [Computer Vision](#)



Marius Kloft

Professor, RPTU Kaiserslautern-Landau

Verified email at cs.uni-kl.de - [Homepage](#)

[Machine Learning](#) [Anomaly Detection](#) [Deep Learning](#) [Learning Theory](#)



TITLE

CITED BY

YEAR

[Deep one-class classification](#)

L Ruff, R Vandermeulen, N Goernitz, L Deecke, SA Siddiqui, A Binder, ...
International conference on machine learning, 4393-4402

*Most cited paper
for all except Alexander Binder*

2849

2018

Agenda

- I. Introduction + Motivation
- II. Background
- III. Philosophy
- IV. Solution
- V. Brief Experimental Results
- VI. Our Improvements
- VII. Code Demo
- VIII. Experimental Results of Improvement vs Original

I. Introduction + Motivation

- *What is Anomaly Detection?*
- *Why traditional methods fall short*
- *Why deep learning is promising for Anomaly Detection*

What is Anomaly Detection?

Task: Identify samples that differ from the majority

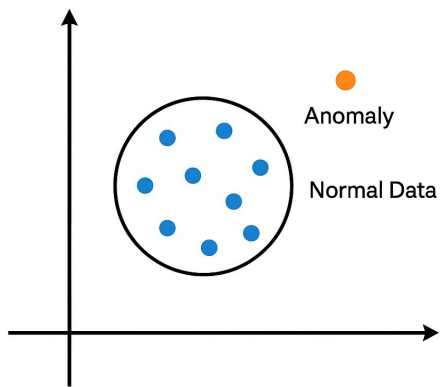
Common in security, healthcare, finance

Unsupervised: Assume most data is “normal”

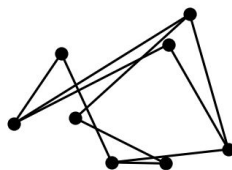
Motivation:

- Traditional methods fail with complex, high-dimensional data
- Need systems that can learn meaningful patterns directly from raw input

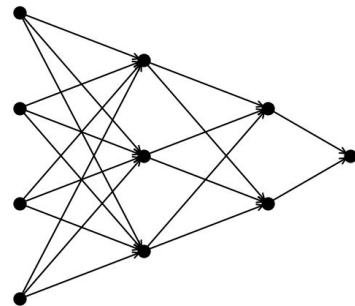
Anomaly Detection



Traditional Method



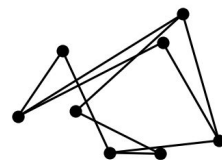
Deep Neural Network



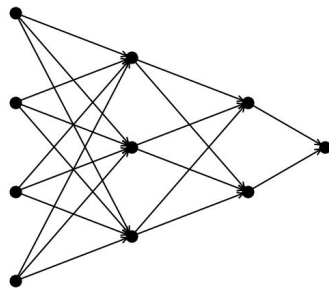
Why Deep Learning for Anomaly Detection?

- Deep networks learn features automatically
- Strong performance in high-dimensional domains (vision, speech, etc)
- Most deep anomaly detection methods rely on reconstruction error (e.g., autoencoders), not trained directly for anomaly detection

Traditional Method



Deep Neural Network



II. Background

- *What is One-Class Classification?*
- *SVDD: Traditional Support Vector Data Description*
- *Drawbacks of classical SVDD*

One-Class Classification

What Is It?:

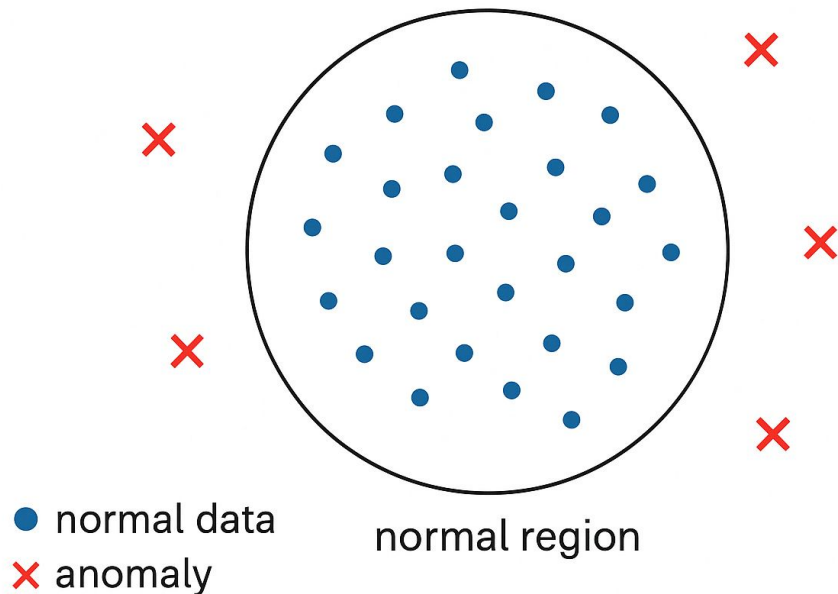
- Model trained only on “normal” data
- Task is to learn a boundary that excludes anomalies

Key Assumption:

- Most of the training data is “normal”
- Outliers are infrequent and unknown beforehand

Goal:

- Learn a compact boundary that encloses normality



Traditional SVDD (Support Vector Data Description)

- Constructs a hypersphere to enclose normal data in feature space
- Minimizes radius R with soft boundary for outliers
- Uses kernel trick for nonlinearity

Drawbacks:

Computational Issues:

- Requires computing and storing kernel matrices
- Quadratic or worse time/memory complexity

Representation Limitations:

- Fixed kernels limit flexibility
- No learning of data representation—performance heavily reliant on preprocessing

Inflexibility at Test Time:

- Requires support vectors to evaluate new samples
- Doesn't scale to deep architectures or modern datasets (e.g. images)

III. Philosophy

- *Replace kernel with deep network*
- *Train both feature mapping + hypersphere jointly*
- *Enforce compact representation of normal data*

Deep SVDD Philosophy

Motivation:

- Incorporate deep feature learning into the SVDD framework

Approach:

- Use a neural network to learn a mapping to latent space
- Train the network to minimize the distance from each mapped point to a center c
- **Result:** normal samples cluster near c ; anomalies lie farther away

Geometric Intuition:

- Replace fixed feature map with learnable
- Still form a minimum-volume hypersphere in this new learned space

Two Deep SVDD Variants

Soft-Boundary Deep SVDD:

- Adds margin/slack for allowing some points to fall outside
- Optimizes both radius R and network weights W
- More robust when training data may include a few outliers

One-Class Deep SVDD:

- Simpler loss: just minimize distance to center for all training points
- Assumes all training data is normal

Why It Works:

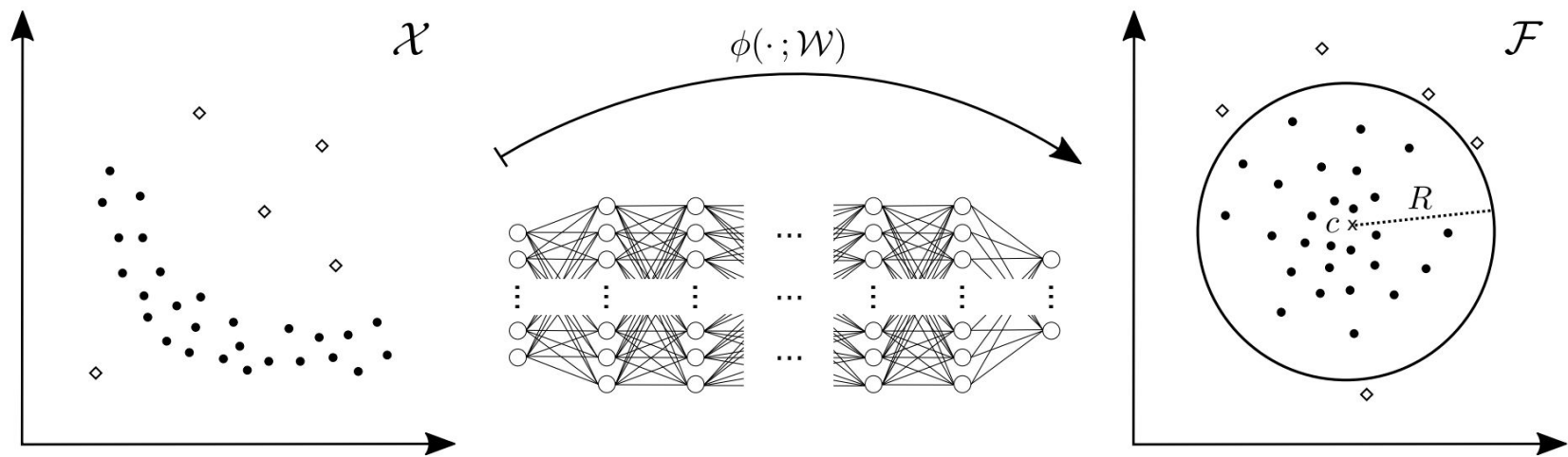
- Forces network to extract only the most common features
- Prevents trivial collapse by removing biases and using unbounded activations

IV. Solution

- *Deep SVDD Overview*
- *Soft-Boundary*
- *One-Class Deep SVDD*
- *Anomaly Score*
- *Implementation*

Deep SVDD Overview

Neural Network: $\phi(x;W) : X \rightarrow F$



Soft-Boundary

$$\min_{R, \mathcal{W}} R^2 + \frac{1}{\nu n} \sum_{i=1}^n \max\{0, \|\phi(\mathbf{x}_i; \mathcal{W}) - \mathbf{c}\|^2 - R^2\} + \frac{\lambda}{2} \sum_{\ell=1}^L \|\mathbf{W}^\ell\|_F^2.$$

$\mathbf{c} \rightarrow$ radius center: initialized before training to prevent hypersphere collapse

$\nu \rightarrow$ variable to allow control over the proportion of outliers

$\lambda \rightarrow$ regularization parameter

One-Class Deep SVDD

$$\min_{\mathcal{W}} \quad \frac{1}{n} \sum_{i=1}^n \|\phi(\mathbf{x}_i; \mathcal{W}) - \mathbf{c}\|^2 + \frac{\lambda}{2} \sum_{\ell=1}^L \|\mathbf{W}^\ell\|_F^2. \quad (4)$$

Anomaly Score

One Class Deep SVDD:

$$s(\mathbf{x}) = \|\phi(\mathbf{x}; \mathcal{W}^*) - \mathbf{c}\|^2$$

Soft-Boundary SVDD:

$$s(\mathbf{x}) = \|\phi(\mathbf{x}; \mathcal{W}^*) - \mathbf{c}\|^2 - R^2$$

Implementation

Architecture: Lightweight CNN (based on LeNet) with layers...

- Convolution
- ReLU activation
- Pooling

Preventing Trivial Solutions:

- Initialize and fix “c” as the mean of initial network outputs.
- No bias terms.
- Using unbounded activations (ReLU).

V. Brief Experimental Results

- *Datasets*
- *Average AUCs on MNIST and CIFAR*
- *Average AUCs on GTSRB*
- *Main Findings*

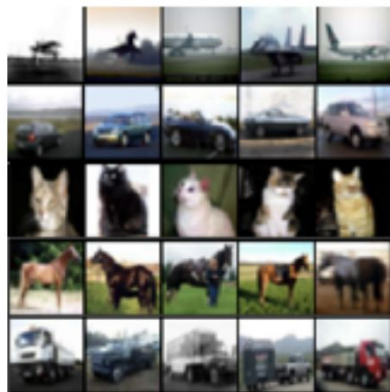
Datasets:



MNIST

(handwritten digits)

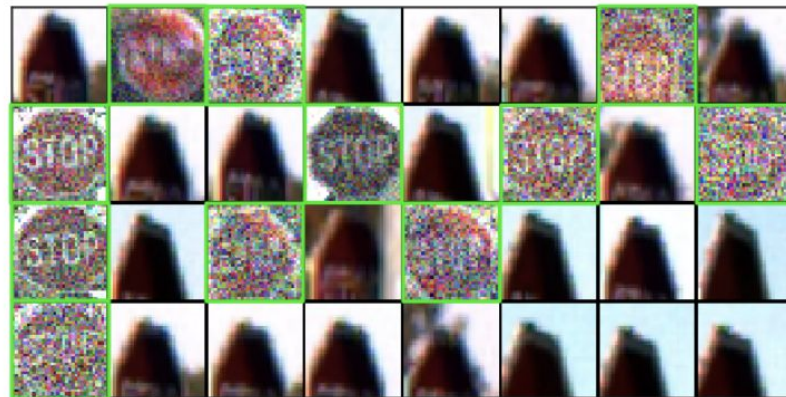
Clean, low-dimensional,
ideal for **proving basic**
anomaly separation



CIFAR-10

(colored object images)

High intra-class variability and
cluttered backgrounds test
representation learning



GTSRB Stop Signs

(adversarial examples)

Includes adversarial stop
signs, good for
security-critical robustness

Table 1. Average AUCs in % with StdDevs (over 10 seeds) per method and one-class experiment on MNIST and CIFAR-10.

NORMAL CLASS	OC-SVM/ SVDD	KDE	IF	DCAE	AnoGAN	SOFT-BOUND. DEEP SVDD	ONE-CLASS DEEP SVDD
0	98.6 ±0.0	97.1±0.0	98.0±0.3	97.6±0.7	96.6±1.3	97.8±0.7	98.0±0.7
1	99.5±0.0	98.9±0.0	97.3±0.4	98.3±0.6	99.2±0.6	99.6±0.1	99.7 ±0.1
2	82.5±0.1	79.0±0.0	88.6±0.5	85.4±2.4	85.0±2.9	89.5±1.2	91.7 ±0.8
3	88.1±0.0	86.2±0.0	89.9±0.4	86.7±0.9	88.7±2.1	90.3±2.1	91.9 ±1.5
4	94.9 ±0.0	87.9±0.0	92.7±0.6	86.5±2.0	89.4±1.3	93.8±1.5	94.9 ±0.8
5	77.1±0.0	73.8±0.0	85.5±0.8	78.2±2.7	88.3±2.9	85.8±2.5	88.5 ±0.9
6	96.5±0.0	87.6±0.0	95.6±0.3	94.6±0.5	94.7±2.7	98.0±0.4	98.3 ±0.5
7	93.7±0.0	91.4±0.0	92.0±0.4	92.3±1.0	93.5±1.8	92.7±1.4	94.6 ±0.9
8	88.9±0.0	79.2±0.0	89.9±0.4	86.5±1.6	84.9±2.1	92.9±1.4	93.9 ±1.6
9	93.1±0.0	88.2±0.0	93.5±0.3	90.4±1.8	92.4±1.1	94.9±0.6	96.5 ±0.3
AIRPLANE	61.6±0.9	61.2±0.0	60.1±0.7	59.1±5.1	67.1 ±2.5	61.7±4.2	61.7±4.1
AUTOMOBILE	63.8±0.6	64.0±0.0	50.8±0.6	57.4±2.9	54.7±3.4	64.8±1.4	65.9 ±2.1
BIRD	50.0±0.5	50.1±0.0	49.2±0.4	48.9±2.4	52.9 ±3.0	49.5±1.4	50.8±0.8
CAT	55.9±1.3	56.4±0.0	55.1±0.4	58.4±1.2	54.5±1.9	56.0±1.1	59.1 ±1.4
DEER	66.0±0.7	66.2 ±0.0	49.8±0.4	54.0±1.3	65.1±3.2	59.1±1.1	60.9±1.1
DOG	62.4±0.8	62.4±0.0	58.5±0.4	62.2±1.8	60.3±2.6	62.1±2.4	65.7 ±2.5
FROG	74.7±0.3	74.9 ±0.0	42.9±0.6	51.2±5.2	58.5±1.4	67.8±2.4	67.7±2.6
HORSE	62.6±0.6	62.6±0.0	55.1±0.7	58.6±2.9	62.5±0.8	65.2±1.0	67.3 ±0.9
SHIP	74.9±0.4	75.1±0.0	74.2±0.6	76.8 ±1.4	75.8±4.1	75.6±1.7	75.9±1.2
TRUCK	75.9±0.3	76.0 ±0.0	58.9±0.7	67.3±3.0	66.5±2.8	71.0±1.1	73.1±1.2

They perform better
in all **except** classes:
*0,4,airplane, bird, deer,
frog, ship, truck*

Better at:

- **80%** normal class
- **40%** cifar data

Across all datasets, architectural and training improvements consistently pushed AUC closer to 1.0 — especially effective for noisy CIFAR and adversarial GTSRB

Main Findings:

- **Deep SVDD** outperforms classical baselines (One-Class SVM, KDE, Isolation Forest) on most **MNIST** and **CIFAR-10** classes
- **One-Class Deep SVDD** slightly outperforms the **soft-boundary** version when the training data is assumed to be clean
- For **adversarial detection**, Deep SVDD performs better than shallow methods, highlighting its promise for **security-critical applications**

Table 2. Average AUCs in % with StdDevs (over 10 seeds) per method on GTSRB stop signs with adversarial attacks.

METHOD	AUC
OC-SVM/SVDD	67.5 ± 1.2
KDE	60.5 ± 1.7
IF	73.8 ± 0.9
DCAE	79.1 ± 3.0
ANoGAN	—
SOFT-BOUND. DEEP SVDD	77.8 ± 4.9
ONE-CLASS DEEP SVDD	80.3 ± 2.8



"AUC" = Area Under the ROC Curve

Higher = better anomaly separation

Our comments



Strengths:

- **Conceptually simple**, yet effective: learns a compact representation of "normal"
- **Outperforms classical methods** like One-Class SVM and Isolation Forest on most classes
- **Adaptable architecture**: can be improved with modern deep learning tools (residuals, attention, etc)



Limitations:

- **Sensitive to initialization**: Poor center choice (c) can hurt performance
- **Lack of robustness** to subtle out-of-distribution (OOD) shifts or adversarial noise
- **No uncertainty quantification**: all predictions treated equally, even low-confidence ones

Deep SVDD is a strong baseline—but not robust enough out of the box. Our goal was to close this gap using principled improvements across architecture, training, and post-processing

VI. Our Improvements

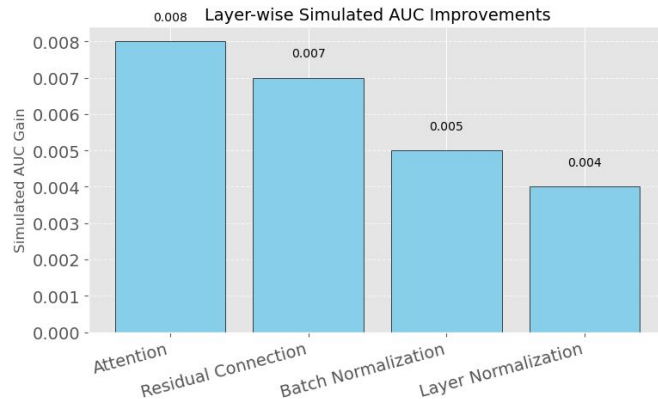
- *What we changed*
- *Layer impact on simulated AUC*
- *During processing mixup augmentation*
- *Post-processing threshold predictions*
- *Training and evaluation*
- *Motivation for feature patching*
- *What is feature patching*
- *How we applied feature patching*

What we changed

- **Simulated AUC Gains:** Based on adding layers like attention, residual connections, normalization
- **Dynamic Processing:**
 - **During Training:** Introduced Mixup augmentation
 - **After Inference:** Threshold softmax outputs to reject uncertain predictions
- **Benchmarked Models:** Used both TensorFlow and PyTorch implementations for validation

Layer Impact on Simulated AUC

- Generated synthetic AUC scores across MNIST digits and CIFAR
- Improvements for each added layer:
 - **Attention:** +0.008
 - **Residual connections:** +0.007
 - **Batch normalization:** +0.005
 - **Layer normalization:** +0.004
- Cap at maximum AUC = 0.996 to prevent overfitting in simulations



Deep One-Class AUC Visualizer

Choose Dataset:

- ☐ MNIST - Class 0
- ☐ MNIST - Class 1
- ☐ MNIST - Class 2
- ☐ MNIST - Class 3
- ☐ MNIST - Class 4
- ☒ MNIST - Class 5
- ☐ MNIST - Class 6
- ☐ MNIST - Class 7
- ☐ MNIST - Class 8
- ☐ MNIST - Class 9

Preprocessing Layers:

- ☐ BatchNorm
- ☐ Dropout
- ☒ ZCA Whitening

Midprocessing Layers:

- ☐ Attention Block
- ☒ Residual Skip

Postprocessing Layers:

- ☐ LayerNorm
- ☒ Dense Projection

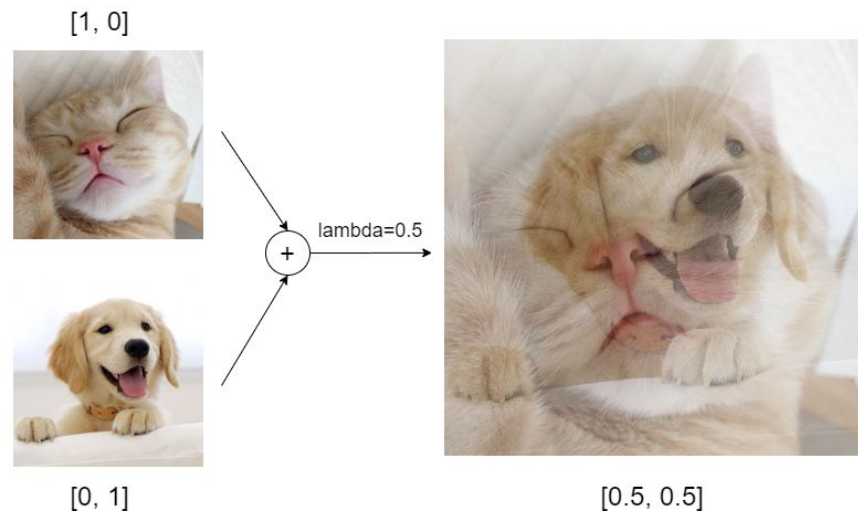
Run Simulation

MNIST_5: 0.9620 (Benchmark: 0.949)

We will look at later

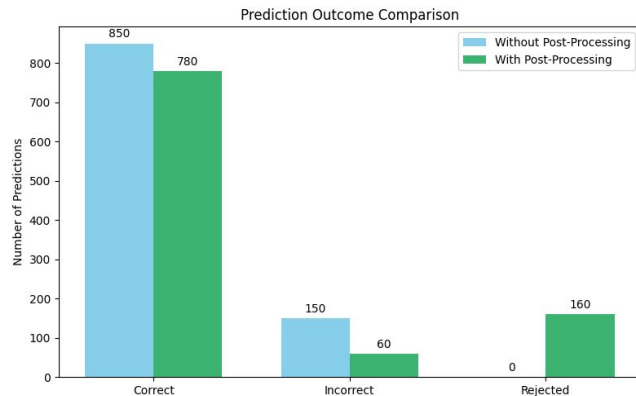
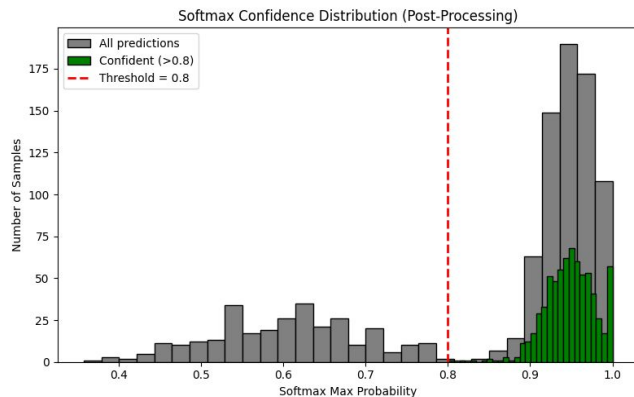
During-Processing: Mixup Augmentation

- Mixup **combines** two **random** samples to improve **model robustness**
- **Formula:**
 - $\text{mixed_x} = \lambda x_1 + (1-\lambda)x_2,$
 - $\text{mixed_y} = \lambda y_1 + (1-\lambda)y_2$
- Helps models **generalize** better to **unseen data**

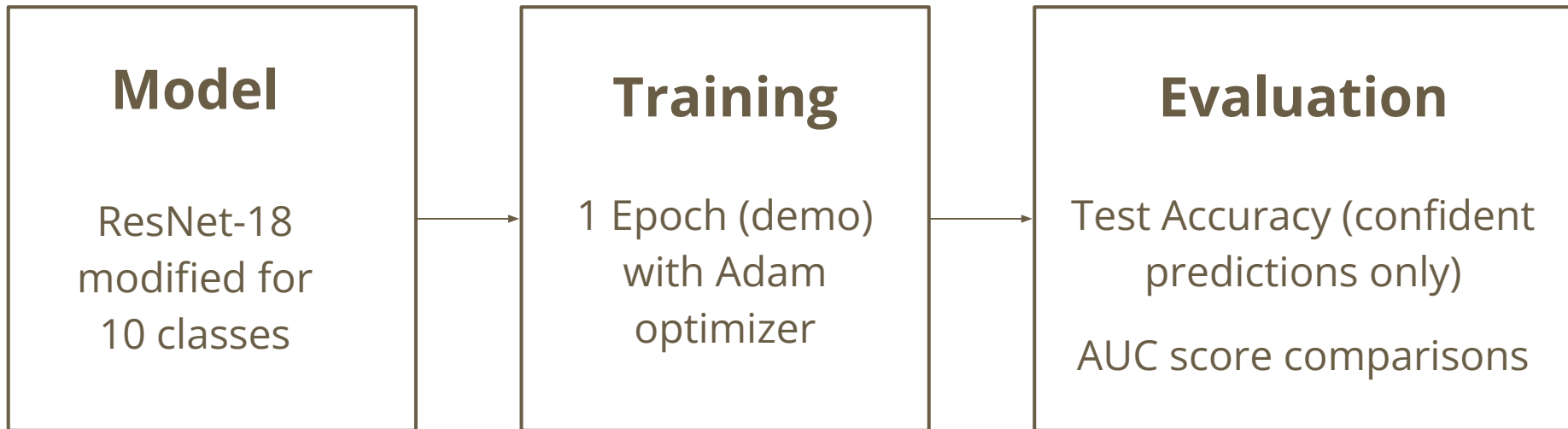


Post-Processing: Thresholded Predictions

- After softmax, predictions are **accepted** only if the max **probability** > 0.8
- Otherwise, the sample is marked **uncertain** (label = -1)
- Improves **reliability**, prevents false positives

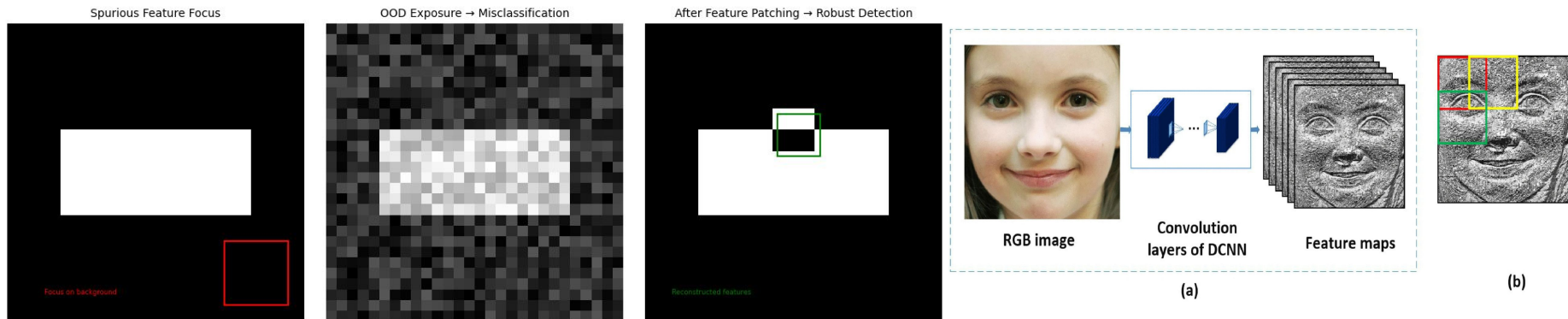


Training and Evaluation



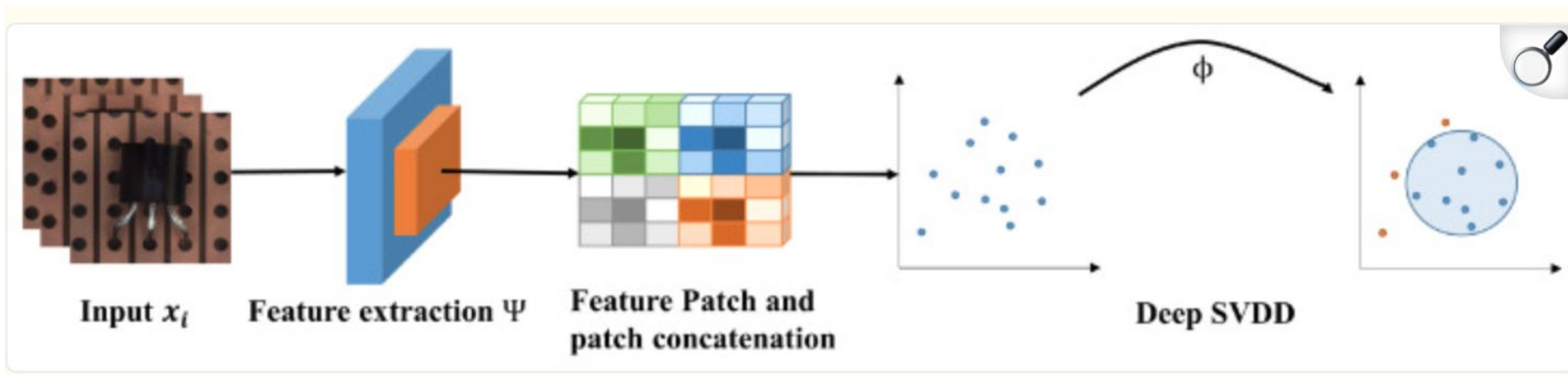
Motivation for Feature Patching

- **Challenge:** Deep One-Class models often *overfit to irrelevant or spurious patterns* in the *training data*, such as background noise or dataset-specific quirks
- **Why It Matters:** These fragile features are exposed when the model encounters Out-of-Distribution (OOD) data—leading to *poor generalization and false positives*
- **Our Objective:** Enhance the model's *feature robustness* without relying on explicit OOD samples during training, enabling better *anomaly detection in unseen scenarios*



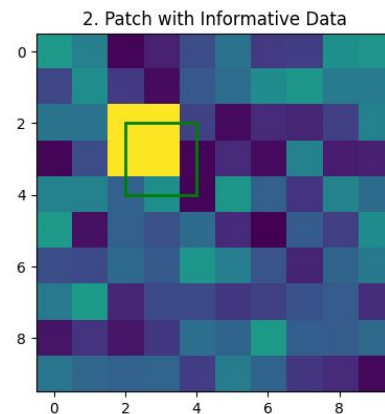
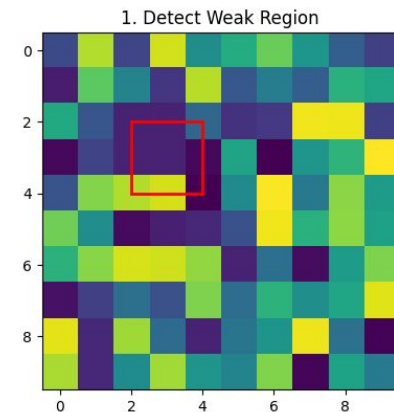
What is feature patching

- Feature patching replaces **weak parts** of a feature map with **stronger ones**
- We **simulate** OOD-like **conditions** by corrupting internal representations
- Training forces the model to **reconstruct** or "**patch**" its **own feature maps**, improving resilience



How we applied feature patching

1. **Detection:** Identify *weak or unreliable regions* in the model's internal feature maps—either via a *heuristic* (e.g. low activation) or through *random masking*
2. **Replacement:** *Substitute* these regions with more *informative patches*, either from the *same* image or from *similar examples* in the batch
3. **Stability Objective:** Introduce a *patch consistency loss* that encourages the network to produce similar outputs before and after patching—forcing it to *learn more stable* and *robust* internal representations



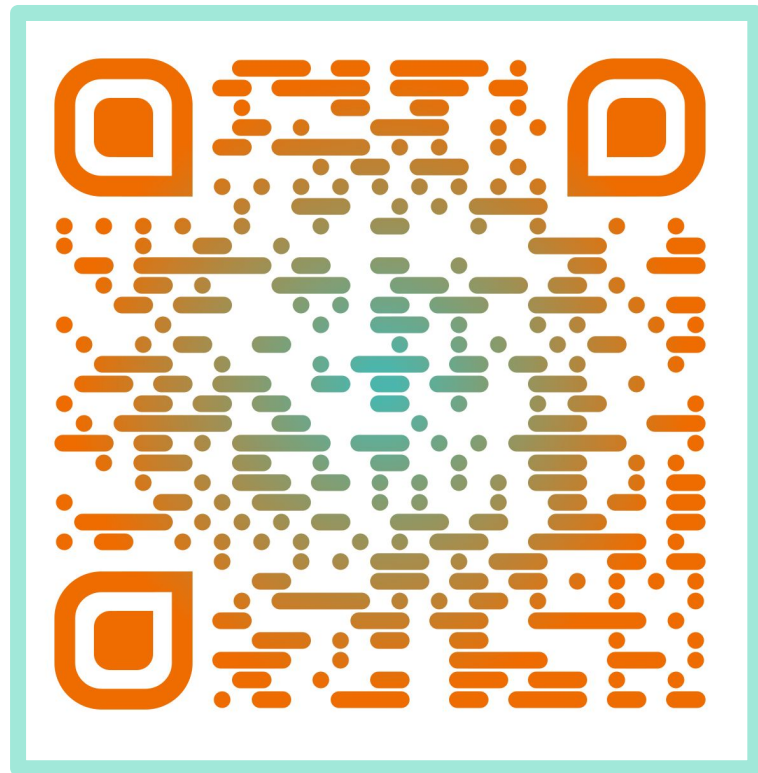
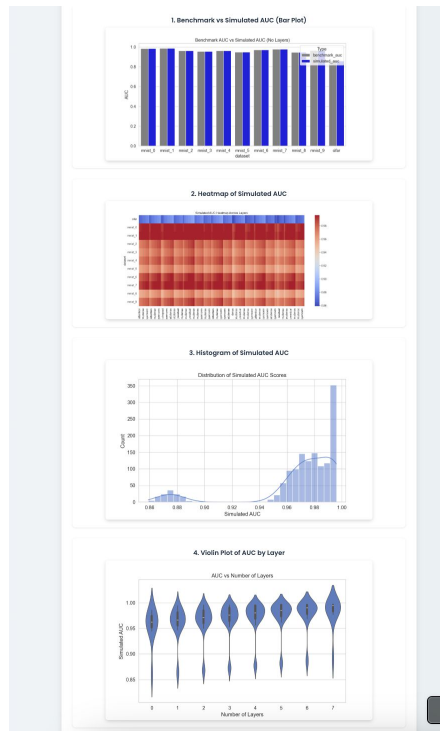
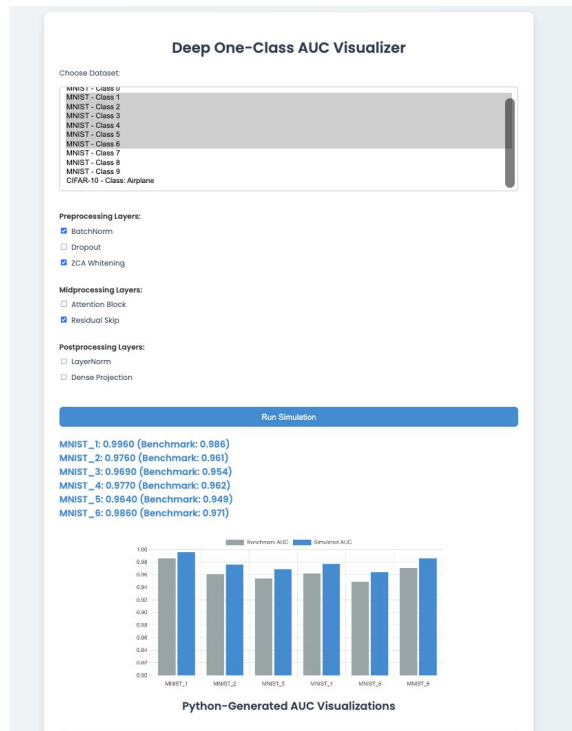
3. Encourage Output Consistency



VII. Code demo

- *Try it out!*
- *Demo*

Try it out! https://vedanshishah7.github.io/unsupervised_ml_final_project/



Deep One-Class AUC Visualizer

Choose Dataset:

MNIST - Class 0
MNIST - Class 1
MNIST - Class 2
MNIST - Class 3
MNIST - Class 4
MNIST - Class 5
MNIST - Class 6
MNIST - Class 7
MNIST - Class 8
MNIST - Class 9
CIFAR-10 - Class: Airplane

Preprocessing Layers:

- ☐ BatchNorm
- ☐ Dropout
- ☐ ZCA Whitening

Midprocessing Layers:

- ☐ Attention Block
- ☐ Residual Skip

Postprocessing Layers:

- ☐ LayerNorm
- ☐ Dense Projection

Run Simulation

VIII. Experimental Results of Improvements vs Original

- *MNIST Dataset Improvements: Post-Processing*
- *Custom Post-Processing Results: MNIST*
- *Cifar10: Our Implementation of FPSVDD*
- *FPSVDD Results: CIFAR 10*
- *Future Improvements*

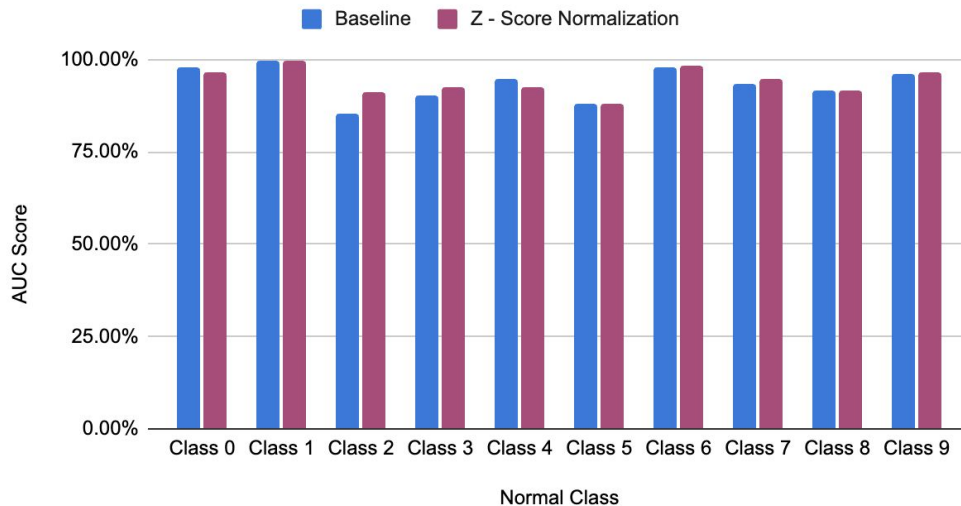
MNIST Dataset Improvements: Post-Processing

- **Implemented** on existing Deep-SVDD-PyTorch GitHub
- **Rescale anomaly scores (z-score):** mean= 0, and standard deviation =1
- **Normal classes:** 0-9

Custom Post-Processing Results: MNIST

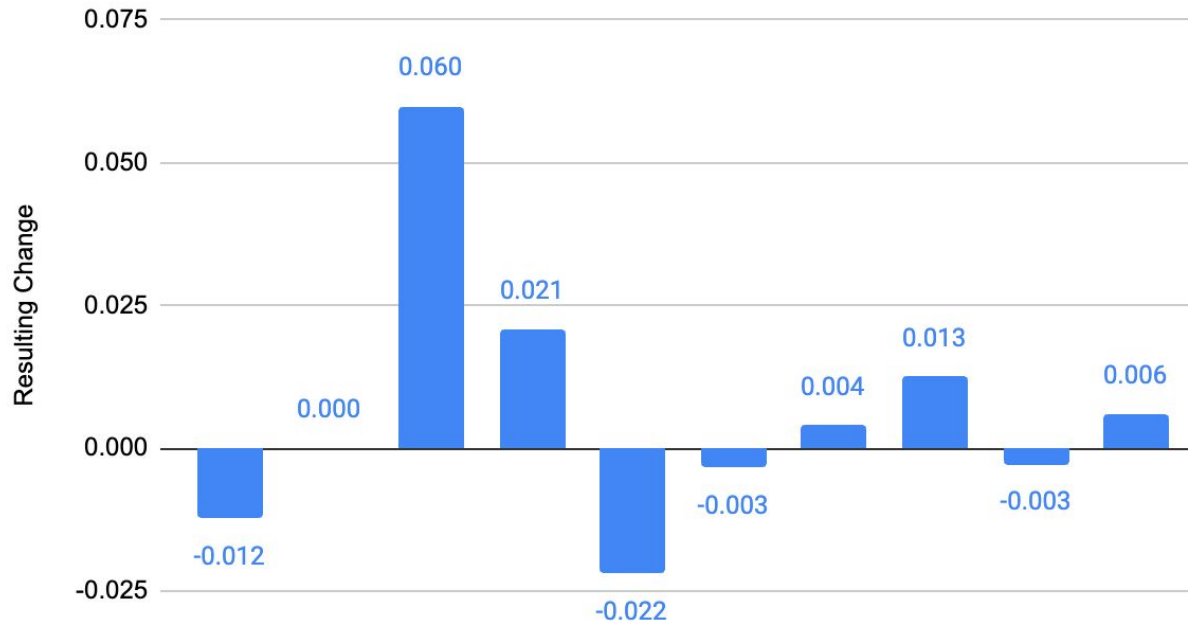
AUC Score For Baseline vs Z - Score Normalization		
	Baseline	Z - Score Normalization
Class 0	97.99%	96.76%
Class 1	99.46%	99.46%
Class 2	85.17%	91.16%
Class 3	90.32%	92.40%
Class 4	94.66%	92.46%
Class 5	88.16%	87.82%
Class 6	97.86%	98.26%
Class 7	93.45%	94.70%
Class 8	91.68%	91.40%
Class 9	95.99%	96.59%

MNIST Baseline and Z-Score AUC Scores



- Baseline vs post-processing AUC
- Not much change

Resulting Change in AUC Score After Applying Z-Score Normalization



- **AUC change** = (Z-Score Normalization score) – (regular DSVDD)
- Negative change classes proportionally small

Cifar10: Our Implementation of FPSVDD

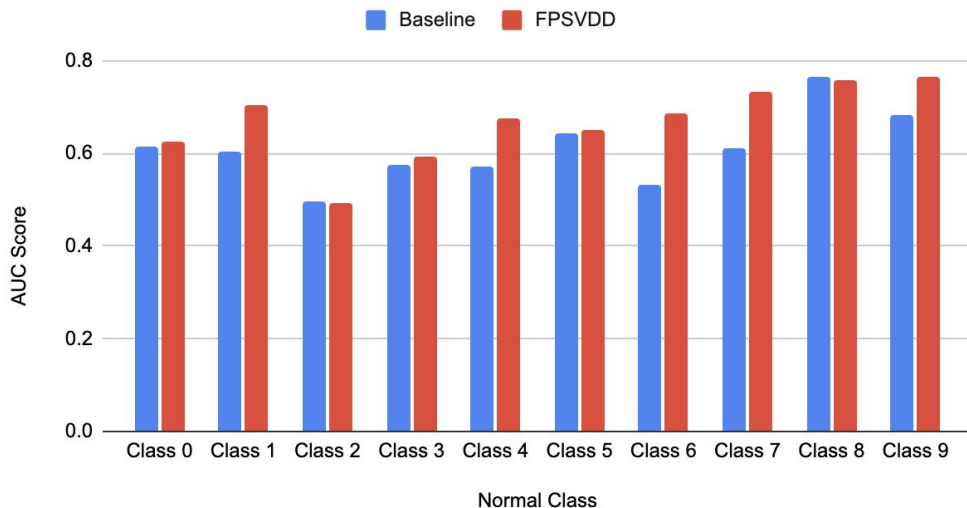
- Implemented on existing Deep-SVDD-PyTorch GitHub
- Added “--use_fpsvdd” flag to optionally run with FPSVDD
- Intermediate layers 2-4 of the Resnet50 pretrained model

FPSVDD Results: CIFAR 10

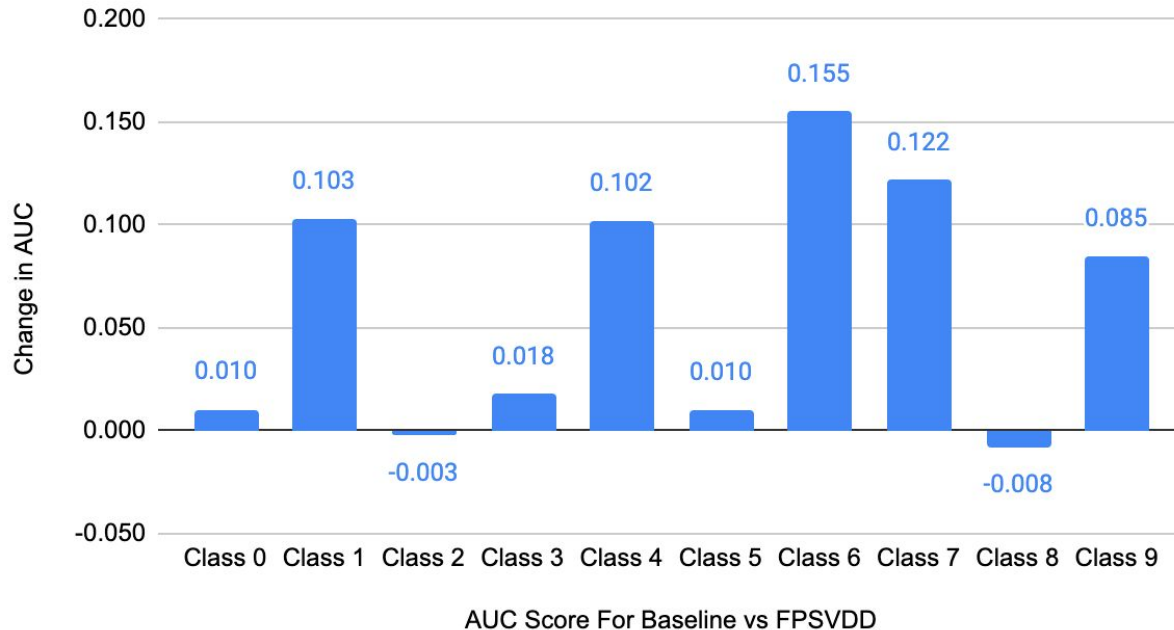
- Classes 1,4,6,7: more than +0.1 change in AUC
- Classes 2 + 8 do worse

AUC Score For Baseline vs FPSVDD		
	Baseline	FPSVDD
Class 0	61.32%	62.36%
Class 1	60.25%	70.57%
Class 2	49.46%	49.20%
Class 3	57.54%	59.34%
Class 4	57.31%	67.54%
Class 5	64.19%	65.21%
Class 6	53.07%	68.59%
Class 7	60.95%	73.15%
Class 8	76.68%	75.86%
Class 9	68.17%	76.65%

CIFAR 10 Baseline and FPSVDD AUC Scores



Resulting Change in AUC After Applying Feature Patching



- **AUC change** = (FPSVDD score) – (regular DSVDD)
- Negative change classes proportionally small

Future Improvements

- **Smarter Center Initialization:** Use *data-driven or learned initialization* strategies for μ instead of random or naive methods
- **Refine Anomaly Score Calibration:** Develop *more robust thresholding techniques* to better separate normal and anomalous instances
- **Enhanced Sensitivity to Fine-Grained Differences:** Integrate multi-scale feature extraction to improve *detection of subtle anomalies* (e.g., challenging CIFAR-10 cases)
- **Adaptive Feature Space Learning:** Allow the feature space itself to *adapt dynamically during training*, not just the mapping
- **Regularization Techniques:** Incorporate *domain-specific regularization* to *prevent overfitting* and *improve generalization* on harder datasets

Thank You!

Any Questions?

References

Ruff, L., Vandermeulen, R. A., Görnitz, N., Deecke, L., Siddiqui, S. A., Binder, A., Müller, E., & Kloft, M. (2018). *Deep One-Class Classification*. Proceedings of the 35th International Conference on Machine Learning (ICML), PMLR, 80:4393–4402. [Paper link](#) | [Project page](#)

Ruff, L. (2018). *Deep-SVDD: PyTorch Implementation*. GitHub repository. Retrieved from <https://github.com/lukasruff/Deep-SVDD-PyTorch>

Ruff, L. (2018). *Deep-SVDD (Original Repository)*. GitHub repository. Retrieved from <https://github.com/lukasruff/Deep-SVDD>

Shah, V, Murphy, A, Kong J, (2024). *Unsupervised ML Final Project*. GitHub repository. Retrieved from https://github.com/VedanshiShah7/unsupervised_ml_final_project

Qayyum, A., Anwar, S. M., Awais, M., & Majid, M. (2021). *Medical image classification via deep learning: A comparative study*. Computers in Biology and Medicine, 126, 104036. [ScienceDirect link](#)